KERN

Infrastructure for distributed energy.

Grid-Serving Storage

Interoperability Profile for Distributed Flexibility

TEB AI Hackathon 2026

Riddle&Code GmbH — Vienna

Jürgen Eckel · Thomas Fürstner

The grid has a coordination problem.

Renewable build-out and electrification loads are stressing distribution grids. Grid expansion takes years. Curtailment is the only fast lever the DSO has.

Meanwhile, distributed battery storage is being deployed at scale — residential through commercial. It sits idle from the grid's perspective.

The capacity exists. The interface does not.

Thousands of storage systems. Dozens of HEMS manufacturers. No standardized way for an aggregator to coordinate them.

What we built.

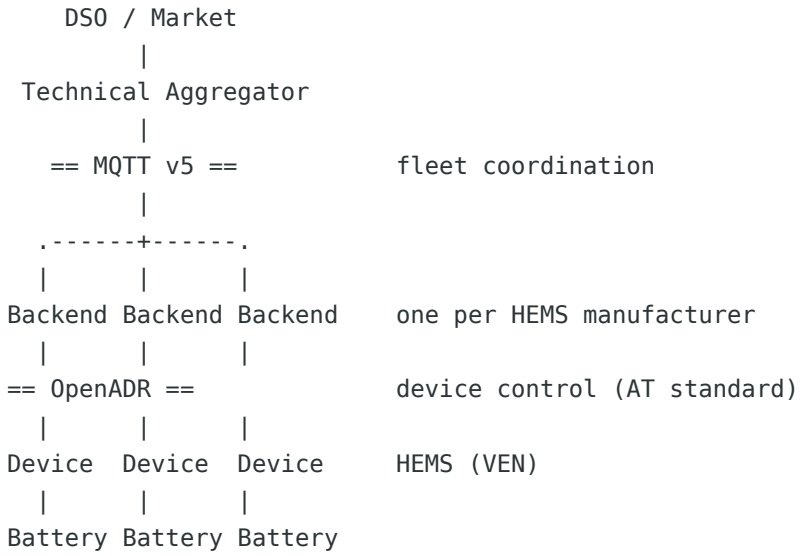
An **interoperability profile** for the use case “Grid-Serving Storage” — designed using the IES methodology, built on the Austrian system architecture for flexible assets (Smart Grids Austria / AIT).

Not a product pitch. Not a business plan. A **rough interoperability profile** — concrete enough to guide implementation, reusable across DSOs, device manufacturers, and aggregators.

Seven documents. One working demo.

The architecture.

Two interfaces. Two protocols. One clean boundary.



MQTT v5 upstream: pub/sub for multi-vendor backend coordination. **OpenADR 2.0b / 3.1** downstream: the Austrian standard for flexible assets. The **HEMS Backend** bridges between them.

How it works.

Four phases. Continuous loop.

Phase	Direction	What happens
Observe	Bottom-up	Backends publish state snapshots and flexibility offers
Plan	Internal	Aggregator runs portfolio MILP
Dispatch	Top-down	Aggregator sends goals. Backends distribute to devices.
Settle	Bottom-up	Delivery reports. Reliability scoring.

The aggregator observes continuously — not just when a grid event occurs. This enables proactive market positioning: mFRR, intraday, congestion management, flex markets.

One dispatch cycle. Many markets. Only the trigger differs.

The optimization.

Two-level hierarchical MILP. Real mixed-integer linear programming.

Level 1 — Aggregator MILP. Treats each backend as an opaque flexibility envelope. Allocates the grid obligation across backends. Penalizes unreliable backends. Respects ramp rates, duration limits, rebound.

Level 2 — Backend MILP. Distributes the received goal across individual devices. Respects per-device SoC, battery limits, prosumer preferences. Translates to OpenADR events.

Why two levels.

- **Privacy.** The aggregator never sees per-device data.
- **Scalability.** Solving for N backends, not N $\times$ M devices.
- **Autonomy.** Each manufacturer optimizes their own devices.

Prosumer sovereignty.

The device has the last word. Always.

The aggregator sends a **soft flex-request** with bounds. Each HEMS device evaluates locally — SoC, hardware limits, prosumer preferences — and opts in or opts out. OpenADR native.

Every failure mode defaults to normal prosumer operation. No persistent standing orders. The aggregator must actively request and actively release.

The HEMS device is the trust boundary.

Market reach.

The same protocol stack serves all Tier 2 and Tier 3 markets.

Market	Response time	Fit
mFRR (tertiary balancing)	12.5 min FAT	Primary
Congestion management	Minutes	Primary
Non-frequency ancillary services	Minutes	Primary
Intraday / day-ahead	Minutes to hours	Reachable
Local flex markets	Variable	Reachable
Dynamic tariff response	Hours	Reachable

Out of scope. Tier 1 (FCR, aFRR) requires SCADA/PLC hardware with sub-second control loops. Different architecture. Different stack.

The demo.

Live simulation of the full Observe → Plan → Dispatch → Settle cycle.

Three simulated HEMS backends: **kern-01** (47 residential), **solar-02** (12 commercial), **flex-03** (25 mixed).

Component	Status
MQTT broker + message flow	**Real** — live Mosquitto
Aggregator MILP (portfolio)	**Real** — PuLP + HiGHS
Backend MILP (per-device)	**Real** — PuLP + HiGHS
Browser dashboard	**Real** — Chart.js + MQTT websockets
Device state, OpenADR exchanges	**Simulated**

`docker compose up --build` then open `dashboard.html`

Or: `python3 run_demo.py` — 50-second terminal walkthrough.

What we delivered.

Interoperability profile — 7 documents, IES methodology:

1. Use case — business context, triggers, serviceable markets
2. Actors & roles — aggregator-centric Austrian model
3. Process flow — four-phase loop, sequence diagrams, edge cases
4. Information model — message types, OpenADR 2.0b + 3.1 mappings
5. Communication — MQTT topic structure, QoS, security
6. Design decisions — 15 decisions with rationale
7. AT vs. DE comparison — Austrian model vs. German SMGW model

Aggregator interface spec — TypeScript types, MILP formulation, aggregation rules.

Working demo — full dispatch cycle with real optimization.

OpenADR 3.1 reference VEN — validated against a real VTN.

Five things to remember.

1. **The interface is the product.**
Standardize the boundary between aggregator and HEMS backends. The rest follows.

2. **Two protocols, one clean boundary.**
MQTT for fleet coordination. OpenADR for device control. The HEMS Backend bridges them.

3. **Two-level MILP preserves privacy at scale.**
The aggregator optimizes across backends without seeing individual devices.

4. **Prosumer sovereignty is non-negotiable.**
Soft signals. Local device autonomy. Fail-safe to normal operation.

5. **One dispatch cycle, many markets.**
mFRR, congestion, intraday, flex markets — same message flow, different trigger.

KERN
Infrastructure for distributed energy.

Riddle&Code GmbH — Vienna

Jürgen Eckel CEO — Profile design, demo, aggregator architecture
Thomas Fürstner CTO — OpenADR 3.1 VEN reference implementation

github.com/RiddleAndCode/grid-serving-storage

juergen@riddleandcode.com

Infrastructure for distributed energy.